

Have a Dice Day!

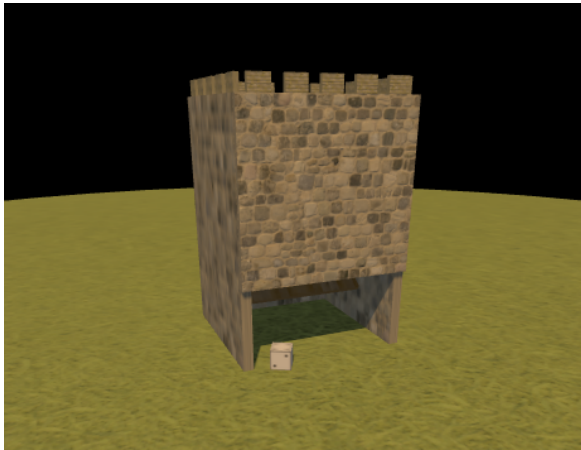
Federico Corsale 1985903

1 Overview and Main Purpose

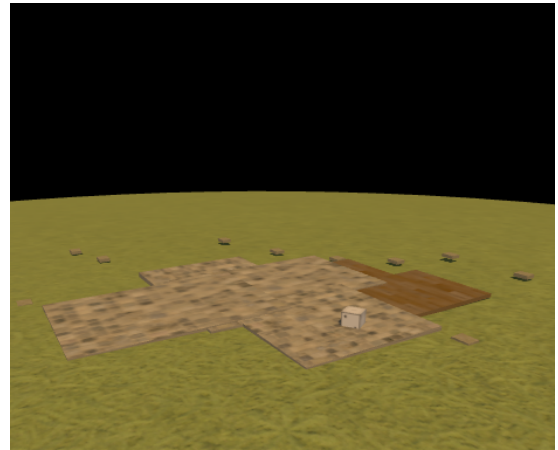
Have a Dice Day! is an interactive 3D application originally developed as the final project for the Interactive Graphics course and later adopted daily during my RPG (role-playing game) sessions.

This application is a Dice Tower simulator inspired by the accessories commonly used in dice-based games. Have a Dice Day! is not a stand alone game, is a visual and interactive tool that allows users to drop a D6 (the standard six-sides dice) through a customizable castle tower. Only a true RPG player knows how catastrophic it can be to roll a 1 in a critical situation. In Have a Dice Day!, whenever this happens, the entire tower dramatically collapses.

The application includes also a bonus feature, called 'Zargos Mode', that unlocks additional dice types such that users can also drops D8, D12 and D20 through the tower. All the details and implementation of tower model, dice model, collapse animation, customization features and Zargos Mode are described in following sections.



(a) Dice Tower



(b) Tower collapsed

Have a Dice Day! overview

2 Technologies and Library

The project has been developed as a web-based application using JavaScript (ES6 modules), HTML and CSS. Have a Dice Day! is entirely built using two principal libraries (one for graphic, one for the physics engine) and a small secondary library for camera controls.

Three.js

Three.js is the graphics library used to render the entire 3D environment. It provides the scene, the camera, the light, the shadows, textures, dice models and the hierarchical tower model. For the main purpose of the application it's been chosen Three.js over WebGL to make use of built-in geometric primitive and Group features.

Cannon-es

Cannon.es is the physics engine responsible of simulating rigid bodies dynamic: gravity and all the collision between dice and parts of the tower.

OrbitControls

OrbitControls, provided by Three.js examples, allow users to inspect the scene interactively.

HTML and CSS

HTML and CSS has been used to create the user menu, where he can actively interact with the scene and visualize results.

All models have been developed from scratch and not imported from external assets.

3 Application Architecture

The application is composed by an HTML file and two folders JS and TEXTURES:

HTML

- index.html: defines the structure of the web application, including the rendering canvas, the user interface (buttons, menus and result panels) and the CSS styling.

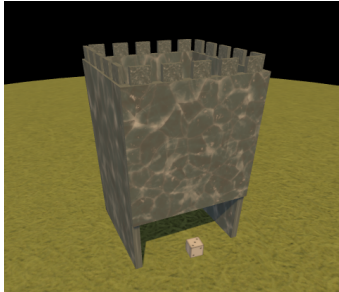
JS

- main.js: is the entry point of the application, it handles the animation loop, the functionality of the user menu and camera controls.
- scene.js: creates the 3D environment, including the lighting system and the floor properties.

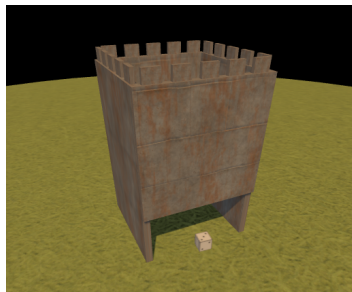
- DiceTower.js: manages the DiceTower class, including its visual three.js build, its physical cannon-es build, animation of collapse, the tower texturing and all class functions.
- dice.js: handles the creation of different dice types. It acts as a factory, allowing main.js to interact with a single dice object without worrying about details of each specific dice.
- d6.js: manages the D6 class, including constructor (cannon build + three build), the generation of the face textures (pips) and methods used to determinate the result of a roll.
- d8.js - d12.js - d20.js : implement remaining dice classes. They are describe together because they share the same overall structure including constructor (cannon build + three build), the generation of the face textures (without pips) and methods used to determinate the result of a roll. The only difference among them is the three geometry used (Octahedron, Dodecahedron, Icosahedron).

TEXTURES

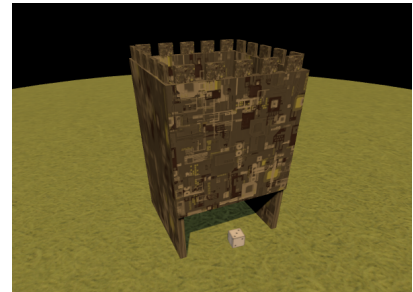
- contains the texture folders grass, rock, stone, wood, ice, rust.
- Each textures folders contains color map, displacement map, roughness map and normals map.



ice tower



rust tower



cyber tower

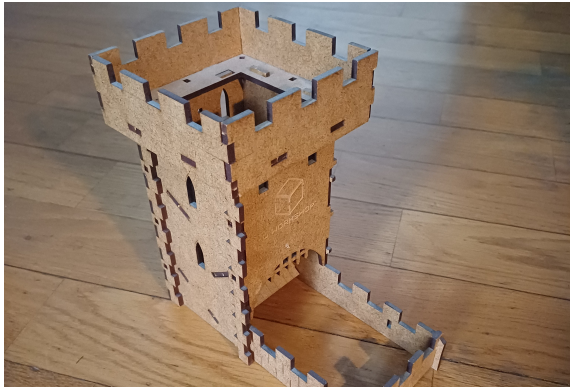
4 Scene, Lighting and Models

Scene and Lighting

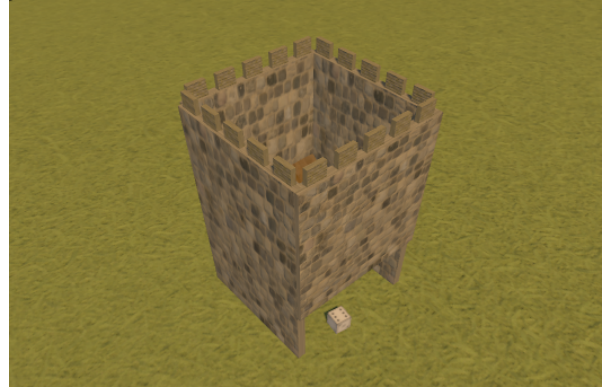
The scene presents an "in/outdoor" environment design to resemble the surroundings of a medieval castle but without a proper sky, this choice has been made to mix an outdoor situation with an indoor one. The floor is a circular textured terrain, circularity gives a better horizon effect than any other shape.

The environment is illuminated by an ambient light that provides uniform illumination to the entire scene and by a directional light that simulates the sunlight, the coloration chosen is sunset oriented (0xffcc88).

Tower Model



(a) Real inspiration



(b) Tower Model

The dice tower is the most complex model, is built as a hierarchical model:

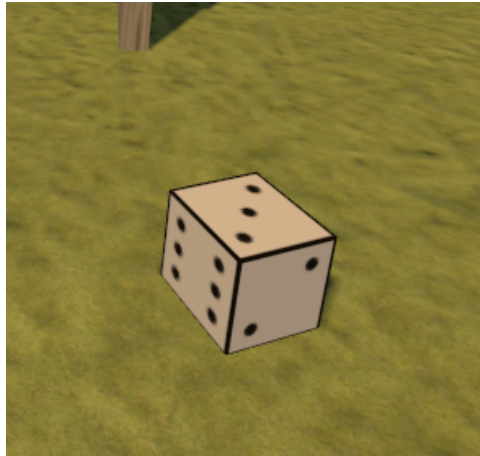
```
towerGroup
|
|-- walls Group
|   |
|   |-- left wall
|   |-- right wall
|   |-- front wall
|   |-- back wall
|
|-- crenel Group
|   |
|   |-- crenel 1
|   |-- crenel 2
|   |-- ...
|   |-- crenel 20
|
|-- ramps Group
|   |
|   |-- ramp 1
|   |-- ramp 2
```

The hierarchical organization is particularly useful during the collapse animation. Initially, the entire tower behaves as a single object during the tower shake. When the collapse is triggered, each child mesh is detached from the structure and animated independently, allowing the walls, crenel and ramps to fall with different velocities and rotations. The walls group and the ramps group are entirely rebuilt in cannon-es as rigid body, crenel aren't physicist object because dices are never suppose to hit them.

Each group is assigned a default texture, which can later be changed through the customization menu. In the default medieval theme, the walls use a stone texture, the ramps use a wood texture and the crenels use a rock texture. In all other customization the same texture is applied to the entire structures (ice, rust and cyber).

D6 model

The D6 model is implemented using a BoxGeometry for the graphical representation and a CANNON.Box for the physical simulation. Each face is assigned a different material containing the corresponding pip pattern, reproducing the appearance of a traditional six-sided die. The mapping between the physical faces and their values is explicitly defined, allowing the application to determine the rolled value by identifying which face is oriented upwards after the die stops rolling.



D6

Getting result

The result of a roll is obtained by comparing the world-space orientation of the six face normals with the world up vector. The face whose normal has the largest dot product with the up vector is considered to be facing upwards.

5 Animation: Shake and Collapse

The tower collapse is entirely implemented in JavaScript without relying on imported animations. The animation is triggered whenever the player rolls a value of 1 with any die.

Shake

Before collapsing, the tower does a shake animation. The effect is obtained by continuous rotating of root node (Tower Group) around axes X and Z acting like one object and creating the impression of an unstable structure.

Collapse

After the shaking phase, every element of the tower is animated independently. Each wall, ramp and crenel is assigned a random initial linear and angular velocity. A simplified

physics simulation is then applied by introducing gravity, causing every piece to fall toward the ground and bounce reducing angular velocity until they stop setting in a specified orientation on the ground.



Tower collapsing

All the scene is reset by dropping a new die through the tower. Whole animation is performed by visual part of the tower, the physical boxes aren't animated. It's only a visual effect.

6 Zargos Mode

Zargos Mode is a bonus features developed for my own use. Originally just the D6 should have been implemented but when I start to use Have a Dice Day! in my personal role-playing session I needed to develop more dice types. In this mode the D8, D12 and D20 are new dice type that can be drop trough the tower.

D8, D12 and D20 models

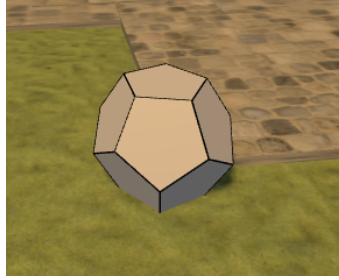
All Zargos mode new dice types are implemented in the same way, each die is represented by its corresponding Three.js polyhedral geometry (OctahedronGeometry, DodecahedronGeometry and IcosahedronGeometry), the funyions computeNormalFace, getTopFace and GetValue are the same of D6 class.

Mode's limitation

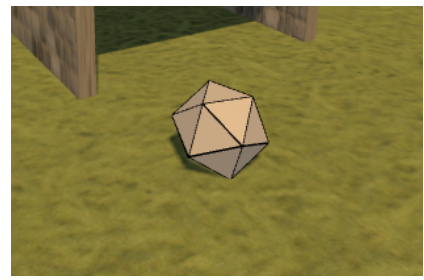
A limitation of the current implementation concerns the visual numbering of the polyhedral dice. Unlike the D6, whose six square faces allow the use of six different materials (one for each face), the D8, D12 and D20 are built from triangular or pentagonal meshes that do not support the same straightforward approach. The only way is to export a pre-fabricated model to use in this cases but the entire dice logic would have been completely rebuild, in particular case main.js in its purpose to reference a single object dice without caring of the type. So this dices appear without numbered faces.



D8



D12



D20

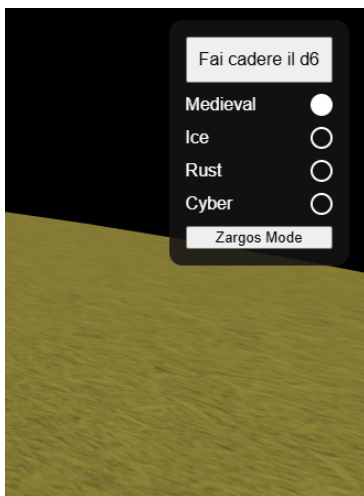
7 A general User Guide

Interaction guide with all User Menu staff:

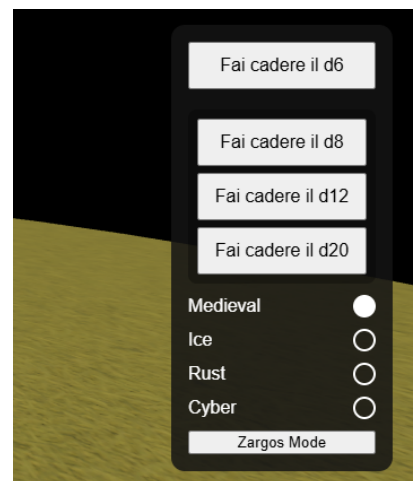
Menu Buttons (top right)

- "Fai cadere il D6" : drop a new D6 through the tower, if the dice tower is collapsed it entirely restores.
- Medieval/Ice/Rust/Cyber : They're four radio button that can be selected to customize the dice tower texture, standard selection is Medieval.
- "Zargos Mode": expand the menu with three new buttons
 - : "Fai cadere il D8" : drop a new D8 through the tower.
 - : "Fai cadere il D12" : drop a new D12 through the tower.
 - : "Fai cadere il D20" : drop a new D20 through the tower.

pressing again the Zargos Mode button will restore the standard menu.



(a) Standard Menu



(b) Menu in Zargos mode

Analist Boxes (top left)

- Value Box : at each drop is possible to visualize the result of the roll. During the fall through the tower appears the sentence "Rullo di tamburi..." that will be replaced with the dice result whenever the dice stop at the bottom.
- History Box: this box collects all previous roll results made during this session in this form "DN: RESULT" where DN and RESULT are the dice type and numeric result of that roll.

8 Future of Have a Dice Day!

Have a Dice Day! started as a university project for the Interactive Graphics course, but during its development it gradually became a tool that I began using in my own role-playing game sessions. This practical use influenced development and the introduction of additional features, such as the Zargos Mode, which extends the application with extra dice commonly used in tabletop RPGs.

Nowadays some features make hard using the application on smartphone. The natural next step for Have a Dice Day! would be a mobile friendly version such that this application would be easily to use everywhere from my smartphone.